2-dars. Date. No.

3. High Threshold (0.8) => precision_high

```
high_thr = 0.8
y_pred_high = (probs >= high_thr).astype(int)
print("\n=== High Threshold (0.8) ===")
print("Predictions:", y_pred_high)
print("Precision:", precision_score(y_true, y_pred_high))
print("Recall:", recall_score(y_true, y_pred_high))
```

Model Improvement

- Data based
  - Advanced Missing Value Treatment
- Method based
- Algorithm based
  - Precision - Recall trade-offs
- Tuning based

Advanced Missing Value Treatment

Cell (bosh katak), N/A, Null

- Missing gijmatlar faqat gina bosh katak emas
- Missing gijmatlar:
  - Modelni ozgartiradi
  - Bias (Missing valuelarni notogri boshqarish natijasida malumot yoki model natijasining haqiqiy gijmatdan bir tomonga ogib ketishi)
  - Notogri natija
  - Mantiqsiz natija
  - Stabil bolmagan model

Imputation == Process filling missing values

- Basic statistical methods like mean, median, mode, fixed value, droprows
- Advanced

~ Types of Missing Value Imputation (Advanced Methods)

- Basicdan kora ogli va yaxshiroq ishlaydigan usullar
- Ular datalar orasidagi munosabatdan organadi

- Types of Missing Value Imputation (Advanced Methods):
  - Mathematical-Statistical Imputation (Advanced)
    - KNN Imputer (Similarity-Based Imputation)
    - MICE (Multiple Imputation by Chained Equations)
  - Machine Learning Methods
    - Predictive (Model-Based) Imputation
  - Rule-Based / Domain Knowledge Methods
    - Domain-Aware Imputation
  - Implementation Strategy
    - Imputation inside Cross-Validation (Leakage Prevention)

1 ~ KNN Imputer (Similarity-Based Imputation)

- KNN Imputer featurelarga ko'ra eng o'xshash k ta qiymatlarni topadi va ularning qiymatlari bilan tushurib qoldirilgan qatorlarni to'ldiradi.
- Qachon:
  - O'xshash featurelar bo'lganda
  - O'xshash qiymatlar bo'lganda
  - Mean va mediandan ko'ra yaxshiroq
- Maktabda o'quvchi bo'yini yozib qo'yishni unutib qoldirdi. 5 ta eng yaqin do'stini (bo'yi, yoshi va jinsida)dan kelib chiqib o'rtalarini bo'yi bilan to'ldiramiz

~ KNN Imputer (Analiz) - Similarity-Based Imputation

Biz Advanced Missing Values ni to'ldirganimizda Encoding oldin qilib olishimiz kerak!!!

```
from sklearn.impute import KNNImputer
imputer = KNNImputer(n_neighbors = 5)
df_imputed = imputer.fit_transform(df)
df_imputed = pd.DataFrame(df_imputed, columns = df.columns)
```

→ (Munchoq ⇒)

2 ~ MICE (Multiple Imputation by Chained Equations)

- MICE usuli tushurib qoldirilgan qiymati bor xar bir feature uchun alohida model qurish orqali ularni to'ldiradi
- Xar bir model cycle (loop) ko'rinishida run bo'ladi (bir chekkadan)
- 10 ta shifokor ko'rigida → xar xil javob → umumlashtirish → yaxshiroq natija

Date. No.

~ MICE (Analiz)

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer
from sklearn.ensemble import RandomForestRegressor

imputer = IterativeImputer(estimator=RandomForestRegressor(), max_iter=10)
df_imputed = imputer.fit_transform(df)
df_imputed = pd.DataFrame(df_imputed, columns=df.columns)
```

3~ ML methods: Predictive (Model-Based) Imputation

- Tushirib qoldirilmagan features lar orqali tushirib qoldirilgan (ustun) qiymatlarni topish va ularni to'ldirish uchun model train qilamiz
- Algorithms:
  - Random Forest
  - XGBoost
  - Linear Regression
- Target qiymatni predict qilish bilan bir xil usul

~ Predictive (Model-Based) Imputation

```
from sklearn.ensemble import RandomForestRegressor
for col in df.columns:
    if df[col].isnull().sum() == 0:
        continue
    train = df[df[col].notnull()]
    test = df[df[col].isnull()]
→ X_train = train.drop(col, axis=1)
    y_train = train[col]
    X_test = test.drop(col, axis=1)
    model = RandomForestRegressor()
    model.fit(X_train, y_train)
    # Predict missing values
    df.loc[df[col].isnull(), col] = model.predict(X_test)
```

(to'g'lash)

Date. No.

4~ Rule-Based / Domain Knowledge Methods : Domain-Aware Imputation

- Not math = use domain knowledge
- Missing salary → use (league average × position ratio)
- Missing xG → use (shots × 0.1)
- Missing position_ratio → based on position:
  - FW = 9
  - CM = 6
  - CB = 4

- Rule-Based / Domain Knowledge Methods

```
import numpy as np
import pandas as pd
    # = Domain Rule 1 =
# customer no phone service, no multiple lines
df.loc[df['PhoneService'] == 0, 'MultipleLines'] = 0
    # = Domain Rule 2 =
# if no internet, all internet-related services must be 0
internet_services = [
        'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
        'TechSupport', 'StreamingTV', 'StreamingMovies' ]
for col in internet_services:
    df.loc[df['InternetService'] == 0, col] = 0
    # = Domain Rule 3 =
# Customers with tenure = 0 are usually brand new → MonthlyCharges = 0
    df.loc[df['tenure'] == 0, 'MonthlyCharges'] = 0
    # = Domain Rule 4 =
# If TotalCharges = 0 but user actually has tenure > 0 → recalc TotalCharges
mask_bad_tc = (df['TotalCharges'] == 0) & (df['tenure'] > 0)
df.loc[mask_bad_tc, 'TotalCharges'] = (
    df.loc[mask_bad_tc, 'tenure'] * df.loc[mask_bad_tc, 'MonthlyCharges'])
```

Date. No.

5 ~ Implementation Strategy:

- Imputation Inside Cross-Validation (Leakage Prevention)
- Butun datasetni impute qilmasdan xar bir qismni alohida impute qilish alohida

~ Implementation Strategy (Best Practice)

- Imputation Inside Cross-Validation (Leakage Prevention)

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score
from sklearn.impute import KNNImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
X = df.drop('Churn', axis=1)
y = df['Churn']
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.2, random_state=42)
pipeline = Pipeline([
    ("imputer", KNNImputer(n_neighbors = 5)),
    ("model", RandomForestRegressor()) ])
scores = cross_val_score(pipeline, X, y, cv = 5)
print(scores)
```